Python & Software Engineering — Working Notes

Indentation and Comments **01 & 02**

Most languages decide where a block of code begins and ends using curly braces **{ }**. Python dropped that idea completely and made indentation itself part of the syntax. Four spaces is the accepted standard for one level of nesting, and the interpreter genuinely uses that whitespace to understand which lines belong inside an if, a loop, or a function. It feels strict when you're new to it, but the upside is real — every Python codebase you open looks structurally similar, regardless of who wrote it.

If you mix tabs and spaces inside the same block, Python refuses to guess what you meant and raises an **IndentationError** instead of silently doing the wrong thing. This is one of the few places where Python is deliberately unforgiving, because ambiguous whitespace has caused real production bugs in other languages.

Comments exist for a narrower purpose than most beginners assume. A comment that restates what the line already says (`counter += 1 # increase counter by one`) adds noise, not clarity. A comment earns its place when it explains something the code itself cannot — *why* a threshold was chosen, an edge case being deliberately ignored, or a workaround for a library bug. Python doesn't have a native multi-line comment syntax; a floating triple-quoted string is used instead, which the interpreter evaluates and discards since nothing stores or prints it.

```
if score >= 40:      # business rule: passing threshold
    status = "Pass"

else:

    status = "Fail"
```

```
""" This block is a de-facto multi-line
comment.
Python evaluates it and throws the value away.
"""
```

Try this / interview angle

- What actually happens internally when Python parses inconsistent indentation — why an error and not a silent guess?
- Take a function you wrote recently — do the comments explain *why*, or do they just repeat *what* the line already says?

Functions 03

A function exists to stop you from writing the same logic twice. The moment the same five lines show up in two different places in a script, that block is asking to be pulled out into a function with a name that describes what it does, not how it does it.

Parameters are the variables a function agrees to accept; arguments are the actual values handed over at call time. Python supports three ways of passing them — positional (order matters), default (a fallback value is used if nothing is passed), and keyword (you name the parameter explicitly, so order stops mattering). A function that returns a value is generally easier to test and reuse than one that just prints — `return` hands data back to whoever called it; `print()` just writes to the terminal and gives the caller nothing to work with.

`*args` and `**kwargs` exist for the case where you don't know in advance how many arguments will be passed. `*args` collects extra positional values into a tuple; `**kwargs` collects extra named values into a dictionary. You'll see both constantly in decorators, wrapper functions, and any API designed to be flexible.

```
def calc_net(gross, tax=0.2):
return gross * (1 - tax)  def
describe(*args, **kwargs):
    print(args)      # tuple of positional extras
print(kwargs)    # dict of keyword extras
```

Try this / interview angle

- Why does a function that returns a value compose better with other code than one that only prints?
- When would you reach for `**kwargs` instead of just listing five named parameters?

OOP Foundations 04

A class is a blueprint, not a thing you can use directly — the same way a house blueprint tells you where the rooms and windows go without being a house you can live in. An object is what you get once that blueprint is actually built: it has real values sitting in real memory. Every object created from the same class shares the same structure but can hold completely different data.

Inside a class you'll find two kinds of members: attributes, which are the data (variables) a class carries, and methods, which are the functions defined inside it that operate on that data. The constructor — `__init__` — runs automatically the moment an object is created, and it's where you usually set up the object's starting attributes using the `self` keyword, which simply points to "this particular object" as opposed to any other object built from the same class.

Encapsulation, inheritance, and polymorphism are the three ideas that make OOP actually useful rather than just decorative. Encapsulation bundles data and the logic that operates on it in one place, so state doesn't leak across the codebase unchecked. Inheritance lets a child class reuse a parent class's attributes and methods instead of re-writing them. Polymorphism lets the same method name behave differently depending on which object it's called on — a `speak()` method might mean something different for a `Dog` object than for a `Human` object, and the caller doesn't need to care which one it's holding.

```
class Employee:     def __init__(self,
name, salary):
        self.name = name
self.salary = salary      def
give_raise(self, pct):
self.salary *= (1 + pct)  e =
Employee("Riya", 50000)
e.give_raise(0.1)
```

Try this / interview angle

- If two objects are built from the same class, what exactly do they share, and what stays independent between them?
- Give a real example (outside programming) of inheritance and explain where the analogy breaks down.

File Handling 05

Opening a file hands your program a live connection to something on disk, and that connection has to be closed when you're done — otherwise you risk leaving the file locked, losing buffered writes, or slowly leaking file handles in a long-running program. The `with` statement (a context manager) exists specifically to guarantee that close happens automatically, even if an exception is raised halfway through reading or writing.

The mode you open a file in changes what's allowed: `'r'` for reading (the file must already exist), `'w'` for writing (creates a new file or silently wipes an existing one — easy to lose data here if you're not careful), `'a'` for appending to the end without touching what's already there, and `'x'` for exclusive creation, which fails loudly if the file already exists instead of overwriting it.

```
with open("data.csv", "r") as f:
    records = f.read() # file is guaranteed
closed here, even on error
```

Try this / interview angle

- What actually goes wrong if you open a file without `with` and an exception is raised before you call `.close()`?
- Why is `'x'` mode safer than `'w'` when you never want to accidentally overwrite something?

Exception Handling 06

Not every error in Python can be handled. A `SyntaxError` or an `IndentationError` means the code is structurally broken and can't even be parsed, let alone run — there's nothing to catch. Exceptions are a different category: problems that happen while otherwise valid code is executing — dividing by zero, a missing file, converting `"abc"` to an integer. Because the code is valid, Python gives you a way to anticipate and recover from these.

`try` wraps the risky code. `except` catches a specific exception type and defines what to do instead of crashing. `else` runs only if nothing went wrong in the try block. `finally` runs unconditionally, whether an exception happened or not — the natural place to close a connection or release a resource. `raise` lets you deliberately throw an exception, including your own custom ones.

Catching a bare `except:` with no exception type is a habit worth breaking early — it silently swallows everything, including typos and bugs that have nothing to do with the situation you were actually trying to handle. Catching the specific exception you expect keeps failures visible instead of hiding them.

```
try:
    val = int(data)
except ValueError:
```

```
    val = 0
else:
print("conversio
n succeeded")
finally:
    print("done, no matter what")
```

Try this / interview angle

- Why can a `SyntaxError` never be caught by a `try/except` block?
- What real-world bug have you seen (or can imagine) that a bare `except:` would silently hide?

NumPy 07

A plain Python list stores its elements as separate Python objects scattered across memory, each carrying its own type information and overhead. NumPy's `ndarray` instead stores a fixed-type block of raw numbers contiguously in memory and pushes the actual looping down into compiled C code. The practical effect is that operations on a NumPy array run in a single vectorized pass instead of Python interpreting a loop line by line — which is the entire reason NumPy exists as the foundation under pandas, scikit-learn, and most of the numerical Python stack.

Broadcasting is the mechanism that lets NumPy apply an operation between arrays of different shapes without you writing an explicit loop — adding a single number to every element of an array, for instance, without ever writing `for x in array`. `import numpy as np  marks = np.array([70, 80, 90]) mean_val = np.mean(marks)  # 80.0 scaled = marks * 1.1  # broadcasting, no loop needed`

Try this / interview angle

- Why is a NumPy array faster than a Python list for the same numeric operation, in terms of what's actually happening in memory?
- What's one situation where a plain Python list is still the better choice over a NumPy array?

Pandas 08

Pandas builds on top of NumPy to give you two structures that map naturally onto real data: a `Series` (one labeled column of values) and a `DataFrame` (a table made of several `Series` sharing the same index). Most day-to-day work — filtering rows, grouping and aggregating, merging two datasets on a shared key, cleaning missing values — comes down to a small set of operations you'll use repeatedly: boolean masking, `.groupby()`, `.merge()`, and `.fillna()` / `.dropna()`.

`.loc[]` and `.iloc[]` are the two ways to slice a DataFrame, and mixing them up is a common source of bugs — `.loc` selects by label (the actual index or column name), while `.iloc` selects by integer position, the same way list indexing works.

```
import pandas as pd  df = pd.read_csv("emp.csv")
high_pay = df[df["salary"] > 50000] by_dept =
df.groupby("department")["salary"].mean()
```

Try this / interview angle

- When would `.loc[]` and `.iloc[]` return two completely different rows for the same numeric index value?
- Walk through what `.groupby()` is doing internally, step by step, before it returns an aggregated result.

Selenium Automation 09

Selenium drives an actual browser through the WebDriver API — clicking, typing, scrolling — which makes it the right tool specifically for pages where content is rendered by JavaScript after the initial page load, and a simple HTTP request wouldn't see that content at all.

Timing is the recurring headache with browser automation. An **implicit wait** tells the driver to poll for up to N seconds whenever it can't find an element, applied globally. An **explicit wait** is more precise — you wait for one specific condition (an element becoming clickable, text appearing) before moving on, which is usually the more reliable choice for pages with asynchronous rendering.

It's worth saying plainly: if a site exposes an official REST API for the data you need, use that instead of scraping the rendered page. It's faster, far less brittle against layout changes, and doesn't risk violating a site's terms of service the way scraping sometimes can.

```
from selenium.webdriver.support.ui import WebDriverWait from
selenium.webdriver.support import expected_conditions as EC
element = WebDriverWait(driver, 10).until(
EC.element_to_be_clickable((By.ID, "submit-btn"))
)
```

Try this / interview angle

- Why does an implicit wait sometimes still fail on a page with heavy asynchronous rendering, even when set to a generous timeout?
- Name a scraping task you'd refuse to automate with Selenium and explain why an API (or nothing at all) would be the better call.

Regular Expressions 10

A regular expression describes a *pattern* of text rather than a literal string, and Python's `re` module is what turns that pattern into an actual search, match, or substitution. The core building blocks are worth knowing cold: `\d` matches a digit, `\w` matches a word character, `+` means one or more, `*` means zero or more, and parentheses `( )` capture a group you can pull out separately from the overall match.

Regex is genuinely good at validating formats — emails, phone numbers, IDs — and pulling structured fragments out of otherwise unstructured text. It's a poor tool for parsing anything with real nested structure, like HTML or JSON; a dedicated parser handles nesting correctly, while a regex built to "mostly work" on HTML tends to break the moment the markup gets slightly more complex than the sample you tested against.

```
import re  email_pattern = r"[\w.-
]+@[\w.-]+\.\w+" match =
re.search(email_pattern, text) if
match:
    print(match.group())
```

Try this / interview angle

- Why does a regex-based HTML parser tend to break as markup gets more complex, even if it worked fine on your test sample?
- Write (on paper) a pattern that would match an Indian 10-digit mobile number optionally prefixed with +91.

Multithreading, Multiprocessing and Concurrency vs Parallelism 11–13

Concurrency and parallelism get used interchangeably but describe different things. **Concurrency** means dealing with several tasks over the same period of time, switching between them — one cook alternating between four dishes. **Parallelism** means multiple tasks genuinely running at the same instant — four cooks, each working one dish simultaneously. A single-core CPU can be concurrent but never truly parallel; a multi-core machine can be both.

Multithreading runs several threads inside one process, sharing the same memory space, which makes it a natural fit for I/O-bound work — waiting on a network call or a disk read, where the CPU is mostly idle anyway. In

CPython specifically, the Global Interpreter Lock (GIL) prevents two threads from executing Python bytecode at the exact same instant, which is why threading gives you almost no benefit for CPU-bound work like heavy number crunching.

Multiprocessing sidesteps the GIL entirely by spawning separate processes, each with its own independent memory and its own Python interpreter, letting them genuinely run in parallel across multiple CPU cores. The trade-off is heavier overhead — processes don't share memory automatically, so passing data between them costs more than passing it between threads.

```
from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor

# I/O-bound: many small network calls waiting on a response with

ThreadPoolExecutor(max_workers=8) as pool:     results =

list(pool.map(fetch_url, urls))  # CPU-bound: heavy computation that

needs real parallel cores with ProcessPoolExecutor() as pool:

    results = list(pool.map(crunch_numbers, chunks))
```

Try this / interview angle

- Why does adding more threads barely speed up a CPU-bound loop in CPython, while it clearly helps for a batch of network requests?
- If you were building a web scraper that hits 500 URLs, would you reach for threading or multiprocessing, and why?

SDLC — Software Development Life Cycle 14

SDLC is the structured sequence a piece of software moves through from idea to retirement: requirements gathering, design, coding, testing, deployment, and maintenance. It doesn't prescribe a specific process (Waterfall vs Agile is a separate decision) — it's the underlying map of phases that any process, however it's organized, still has to pass through.

The reason maintenance is on that list and often gets underestimated: most of a system's actual lifetime cost isn't writing the first version, it's the years of bug fixes, security patches, and small feature additions that come after deployment. Skipping proper design or testing early doesn't remove that cost, it just moves it later and makes it more expensive.

Try this / interview angle

- Which SDLC phase, in your experience, tends to get compressed first when a deadline is tight — and what does that usually cost later?

Agile Philosophy 15

Agile is a mindset before it's a set of ceremonies — the core bet is that requirements will change as people actually see working software, so it's better to deliver something small and real early, get feedback, and adjust, rather than spend months designing a complete system upfront that might be wrong by the time it ships.

That translates into a handful of practical habits: short delivery cycles, close collaboration with whoever the software is for, and treating a changing requirement as expected rather than as a failure of planning. Scrum, Kanban, and similar frameworks are specific implementations of this philosophy, not the philosophy itself.

Try this / interview angle

- Explain the difference between Agile as a philosophy and Scrum as a framework — could a team be Agile without using Scrum?

Scrum Framework 16

Scrum is one concrete way to run Agile. Work is broken into a backlog of items, and a fixed-length Sprint (commonly two to four weeks) pulls a chunk of that backlog to actually build. Three roles keep it running: the Product Owner decides what gets built and in what order, the Scrum Master keeps the process itself unblocked, and the Developers do the building.

The ceremonies exist to solve specific coordination problems, not to fill a calendar: the Daily Standup surfaces blockers early instead of letting them sit for a week, the Sprint Review shows real working software to stakeholders instead of a slide deck, and the Retrospective is where the team looks at how it worked together and adjusts — which is often the ceremony teams skip first, and the one that costs them the most in the long run.

Try this / interview angle

- What specific problem does the Daily Standup solve that a weekly status email wouldn't?
- Why does skipping the Retrospective tend to hurt a team more over time than skipping any other ceremony?

Git Version Control 17

Git tracks changes to a codebase as a graph of commits rather than a single linear history, which is what makes branching cheap — you can start a feature branch off `main`, work in isolation, and merge back once it's ready, without ever risking the stable line of history in the meantime.

A Pull Request is the review checkpoint sitting between "I finished this branch" and "this is now part of the main codebase" — it's where a teammate actually reads the diff, runs CI checks, and either approves it or asks for changes before the merge happens. Skipping that step is how small bugs quietly become production incidents.

```
git checkout -b feature-login

git add .

git commit -m "Add login form validation" git push

origin feature-login # open a Pull Request from

feature-login into main
```

Try this / interview angle

- What does a Pull Request actually protect against that pushing straight to `main` doesn't?
- Describe, in your own words, what `git rebase` does differently from `git merge`.

Documentation Standards 18

A README isn't a formality — it's the first thing anyone (including future you) reads before they can even run the project. At minimum it should answer: what does this do, how do I install it, how do I configure it, and how do I actually use it (setup steps, environment variables, and, for anything exposing an API, the interface itself).

Good documentation is written for the reader who knows nothing yet, not for the author who already understands the system — which is exactly why it tends to go stale unless updating it is treated as part of finishing a feature, not an afterthought tackled later.

Try this / interview angle

- Pick a project you've contributed to — could a stranger get it running using only the README, with zero extra context from you?

Risk Management 19

Risk management in software is just structured pessimism, done early enough that it's useful. Every meaningful risk gets scored on two axes — how likely it is to happen, and how bad it would be if it did — and the highest-scoring risks are the ones that get a mitigation plan before they ever become a real incident.

A risk that's high-impact but low-probability (a data center outage) still needs a plan, usually a cheap, standing one like automated backups. A risk that's high -probability but low-impact might just be accepted and monitored rather than engineered around, because the cost of prevention would outweigh the cost of the problem itself.

Try this / interview angle

- Name one risk in a project you're working on right now, and where it would sit on a probability-vs-impact grid.

Coding Standards 20

Coding standards are the agreed rules a team follows for naming, file structure, and error handling so that code written by five different people still reads like it came from one codebase. The value isn't in any single rule being objectively "correct" — it's in everyone following the same one, which is what makes code review fast and onboarding a new teammate less painful.

In a multi-contributor project like CodeSync or a GSSoC repo, this matters more than it does solo — without an agreed standard, every PR turns into a debate about style instead of substance.

Try this / interview angle

- What's one coding-standard disagreement you've actually run into on a team or open-source project, and how was it resolved?

PEP 8 21

PEP 8 is Python's own official style guide, and it's specific rather than vague: 4 spaces per indentation level, a line length capped around 79–88 characters depending on the tool, `snake_case` for functions and variables, `PascalCase` for classes, and `UPPER_CASE` for constants.

Following it isn't about aesthetics — it's about every Python developer being able to read unfamiliar code at normal speed instead of pausing to decode someone's personal naming convention. Tools like Black and Ruff exist largely to enforce PEP 8 automatically, so nobody has to argue about it in code review.

Try this / interview angle

- Name three PEP 8 rules you already follow out of habit, and one you've broken without realizing it.

Docstrings vs Comments 22

A comment is for the person reading the source code right now, inline, explaining a specific line or decision. A docstring is different — it's a `"""triple-quoted string"""` placed right after a module, class, or function definition, and it's meant to document the public interface: what this thing does, what it expects, what it returns.

The practical difference is that a docstring is inspectable at runtime — `help(my_function)` reads it directly, and documentation generators like Sphinx build entire reference sites from docstrings alone. A comment, by contrast, is invisible to anything except someone reading the raw source file.

```
def calc_net(gross, tax=0.2):     """Return take-home pay after
applying a flat tax rate.      gross -- pre-tax amount     tax   --
decimal tax rate, default 0.2     """     return gross * (1 - tax)
# explicit, not compound assignment
```

Try this / interview angle

- Why can a documentation generator build a reference site from docstrings alone but not from inline comments?

Logging and Auditing 23

`print()` is fine for a quick script, but it doesn't scale — there's no way to turn it off in production, no severity level attached to the message, and nothing writing it anywhere durable once the terminal window closes. Python's `logging` module solves all three: messages carry a severity (`DEBUG`, `INFO`, `WARNING`, `ERROR`, `CRITICAL`), and the output can be filtered, formatted, and routed to a file or an external monitoring service without touching the calling code.

One rule that isn't optional: never log passwords, API keys, tokens, or anything else sensitive. Log files often end up with looser access control than the database itself, and a secret that leaks into a log is now effectively public inside the organization.

```
import logging  logging.basicConfig(level=logging.INFO)
logging.info("App booted") logging.warning("Retry limit
```

```
approaching for job %s", job_id) logging.error("Payment
failed", exc_info=True)
```

Try this / interview angle

- Why is a fixed severity level on every log message more useful in production than a plain `print()` statement?
- What's the actual risk if an API key accidentally ends up in a log file, even one nobody reads regularly?

Efficient Code 24

Writing efficient code usually comes down to picking the right data structure before it comes down to clever tricks. Looking up whether a value exists in a Python `list` takes time proportional to the list's size, because Python has to check each element in turn; the same lookup against a `set` or a `dict` key is close to constant time regardless of how large it grows, because both are backed by a hash table.

The habit worth building early is: profile before optimizing. It's tempting to guess which part of a program is slow and rewrite it, but the actual bottleneck is often somewhere unexpected — a database query run in a loop, not the Python logic around it. Tools like `cProfile` tell you where time is really going before you spend effort optimizing the wrong thing.

```
# O(n) lookup — checks every element in the worst case
if user_id in user_list:     ...
 # O(1) average lookup — hash table under the hood
if user_id in user_id_set:     ...
```

Try this / interview angle

- Why does checking membership in a `set` stay fast even as it grows to a million elements, while a `list` doesn't?
- Describe a time you assumed you knew where a slowdown was, and profiling proved you wrong (or how you'd expect that to go).

Architecture Principles — DRY, KISS, YAGNI, SOLID 25

DRY (Don't Repeat Yourself) — the same piece of logic shouldn't exist in two places, because the moment it needs to change, you have to remember to change it everywhere it was copied, and eventually one copy gets missed.

KISS (Keep It Simple) and **YAGNI (You Aren't Gonna Need It)** pull in the same direction — build what's actually required right now, not the flexible, general-purpose version you imagine you might need in six months. Speculative abstraction adds real complexity today in exchange for a benefit that, most of the time, never actually arrives.

SOLID is five design guidelines for object-oriented code: Single Responsibility (a class should have one reason to change), Open/Closed (open for extension, closed for modifying existing working code), Liskov Substitution (a subclass should be usable anywhere its parent class is expected, without surprises), Interface Segregation (don't force a class to implement methods it doesn't need), and Dependency Inversion (depend on abstractions, not on concrete implementations). Together they push toward code that's decoupled enough to test and change one piece at a time.

Try this / interview angle

- Point to one place in a project you've worked on where DRY was violated — what would refactoring it actually look like?
- Give an example of a YAGNI violation — a feature or abstraction built "just in case" that never ended up being used.

Unit Testing 26

A unit test checks one small piece of behavior in isolation — usually a single function — and asserts that a given input produces the expected output, independent of databases, networks, or other parts of the system. The

isolation matters: if a test depends on a live database being in a particular state, it stops being a fast, reliable unit test and becomes something closer to an integration test.

A common pattern worth internalizing is Arrange-Act-Assert: set up the inputs (Arrange), call the function being tested (Act), then check the result (Assert). Tests are typically run automatically as part of a CI pipeline on every push, so a broken change gets caught within minutes instead of surfacing in production days or weeks later. `def add(a, b):  return a + b  def test_add():  # Arrange  a, b = 2, 3  # Act  result = add(a, b)  # Assert  assert result == 5`

Try this / interview angle

- Why does a test that depends on a live external database stop qualifying as a true unit test?
- What breaks first in a fast-moving project if unit tests aren't part of the CI pipeline?

Ruff Linter 27

Ruff is a static analyzer written in Rust, built to be dramatically faster than the older Python-based tools (Flake8, isort, and others) it's largely replaced. It doesn't run your code — it reads it and flags problems: unused imports, dead code, style violations, likely bugs — all before anything ever executes.

Running `ruff check --fix .` both reports issues and auto-fixes the ones that are safe to fix mechanically, which is why it's become a standard pre-commit or CI step — it catches an entire class of small mistakes before a human reviewer even opens the pull request. `ruff check --fix .`

Try this / interview angle

- What's the practical difference between what a linter like Ruff catches and what a unit test catches?

Black Formatter 28

Black is deliberately opinionated — it has almost no configuration options, and that's the entire point. Instead of a team spending time debating tabs vs spaces or where to put a trailing comma, Black just reformats the code the same way every time, for every contributor, and the debate simply stops being available to have.

Running it is a single command, and most teams wire it into a pre-commit hook so code is auto-formatted before it's even committed — which means pull requests show the actual logic change in the diff, not a wall of whitespace noise from someone's editor reformatting the whole file.

```
black .
```

Try this / interview angle

- Why does an opinionated formatter with zero configuration options end up saving a team more time than a flexible one with lots of settings?